SmartCheck 工业视觉检测系统项目框架

- 1. 项目定位
  - 目标: 基于 Qt 5/6 + C++ + OpenCV + Python 的工业视觉检测系统
  - 核心原则: System 与 Business 严格分层, 高内聚低耦合
  - 当前阶段: 先用模拟链路 + SQLite 本地开发, 后续切换 MySQL/真实设备
- 2. 技术栈
  - Qt Widgets: 主界面与信号槽编排
  - Qt Charts: Dashboard 看板, 展示 UPH/合格率/趋势
  - Qt Sql + SQLite: 本地开发数据库
  - C++: 系统层、业务层主控逻辑
  - Python: 推理脚本与模型接口, 当前为 mock_inference.py
  - CMake: 跨模块构建与依赖管理
- 3. 总体分层架构
  - Core 核心契约层
    - 文件: Core/Interfaces.h
    - 内容: ResultCode / DeviceAction / DetectionBox / InferenceResult / ProductionRecord / IConnector
    - 作用: 统一跨模块数据结构和接口边界
  - System 系统基础设施层
    - DeviceLayer 设备层
      - CameraWorker: 异步采图线程, 当前生成模拟图像
      - ModbusConnector: 模拟 PLC 通信, 接收 QC 动作
    - DataStorage 数据层
      - DatabaseManager: SQLite 单例管理, 初始化 schema, 批量写入, 查询统计
      - 未来扩展: 保持接口不变, 后续切换 MySQL + 存储过程
    - SysLog 日志层
      - SysLogService: 异步日志写入, 记录人工待检/异常/动作信息
  - Business 业务逻辑层
    - Inference 推理层
      - PythonBridge: 通过 QProcess 调 Python 脚本
      - InferenceEngine: 解析 JSON, 执行阈值判断, 输出标准结果
    - QC 质控层
      - QCManager: 把推理结果转成质量判定、数据库记录和 PLC 动作
    - Analytics 分析层
      - AnalyticsService: 查询日统计、UPH、合格率, 提供给 UI
  - UI 表现层
    - MainWindow: 只负责展示与装配, 不直接访问数据库或硬件
    - 界面区域: 实时画面 / 当前状态 / KPI 指标 / 柱状图 / 事件列表 / 7 天趋势
- 4. 当前源码目录
  - Core/
    - Interfaces.h
  - System/
    - Device/CameraWorker.*
    - Device/ModbusConnector.*
    - Data/DatabaseManager.*
    - Logging/SysLogService.*
  - Business/
    - Inference/PythonBridge.*
    - Inference/InferenceEngine.*
    - QC/QCManager.*
    - Analytics/AnalyticsService.*
  - UI 入口文件
    - main.cpp
    - mainwindow.h / mainwindow.cpp / mainwindow.ui
  - 辅助资源
    - sql/schema.sql: 本地数据库结构
    - scripts/mock_inference.py: Python 模拟推理脚本
    - record.md: 按分钟记录开发操作
- 5. 核心业务流程
  - 步骤 1: CameraWorker 采集图像
    - 输出: CapturedFrame(frameId, imagePath, preview, capturedAt)
  - 步骤 2: InferenceEngine.process(frame)
    - 调用 PythonBridge 启动 Python 推理脚本
    - 协议: C++ 传 image_path / request_id, Python 返回 JSON
  - 步骤 3: InferenceEngine 解析结果
    - 解析 detection 坐标、类别、confidence、summary
    - 规则: maxConfidence < 0.85 时进入 ManualReview
    - 规则: 高置信度且标签非 ok 时判定 Fail
    - 否则判定 Pass
  - 步骤 4: QCManager.handleResult(result)
    - 生成 ProductionRecord
    - 先写数据库, 再发 PLC 动作
    - Pass -> Accept / Fail -> Reject / ManualReview -> Hold
  - 步骤 5: AnalyticsService.refresh()
    - 查询当日合格、失败、待检总数
    - 计算 yieldRate 与最近 1 小时 UPH
  - 步骤 6: MainWindow 更新展示
    - 更新预览图、状态区、事件列表、KPI 卡片、柱状图、趋势列表
- 6. 关键信号槽关系
  - CameraWorker::imageCaptured -> InferenceEngine::process
  - InferenceEngine::resultReady -> QCManager::handleResult
  - QCManager::actionTriggered -> ModbusConnector::write
  - QCManager::inspectionCompleted -> AnalyticsService::refresh
  - AnalyticsService::snapshotReady / trendReady -> MainWindow
- 7. 数据设计
  - 表 prod_records
    - sn: 产品序列号
    - device_id: 设备编号
    - result_code: 0 合格 / 1 次品 / 2 待检
    - defect_details: 缺陷详情
    - cycle_time: 推理或检测耗时
    - created_at: 记录时间
  - 视图 view_daily_stats
    - 按日期汇总 pass/fail/manual/total
- 8. 当前实现策略
  - 设备模拟优先: 先打通全链路, 后接真实相机和 PLC
  - 数据库双阶段: 现在 SQLite, 后续 MySQL
  - 推理桥接先用 QProcess, 便于替换真实模型脚本
  - UI 与底层隔离, 便于后续做维护和扩展
- 9. 后续演进方向
  - 接入 OpenCV / RTSP 实时相机流
  - 接入真实 Modbus PLC 通信
  - 将 DatabaseManager 切换为 MySQL 实现
  - 补自动化测试: JSON 解析 / 阈值判定 / 批量写入 / 统计聚合
  - 扩展 Dashboard: 班次统计、设备对比、缺陷 Pareto 分析
- 10. 讲解建议顺序
  - 先讲项目目标和分层原则
  - 再讲从 Camera 到 Python 到 QC 到 PLC 的主链路
  - 然后讲数据库记录与 Dashboard 聚合
  - 最后讲当前模拟方案与未来真实产线落地路径

Presented with xmind